- If the middleware detects that the request is probably from a browser (the `Accept` header contains `*/*`), it will *not* use conneg. Instead, it formats the response as though an `Accept` header was not provided, using the default formatter (typically JSON).

These defaults are relatively sane, but they can certainly bite you if you're not aware of them. That last point in particular, where the response to a request from a browser is virtually always formatted as JSON, has certainly caught me out when trying to test XML requests locally!

As you should expect by now, all these rules are configurable; you can easily change the default behavior in your application if it doesn't fit your requirements. For example, the following listing, shows how you can force the middleware to respect the browser's `Accept` header and remove the `text/plain` formatter for `string`s.

Listing 20.14 Customizing MVC to respect the browser's `Accept` header in web APIs

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers(options =>                          ❶
{
    options.RespectBrowserAcceptHeader = true;                      ❷
    options.OutputFormatters.RemoveType<StringOutputFormatter>();   ❸
});
```

❶ AddControllers has an overload that takes a lambda function.

❷ False by default; several other properties are also available to be set.

❸ Removes the output formatter that formats strings as text/plain

In most cases, conneg should work well for you out of the box, whether you're building an SPA or a mobile application. In some cases, you may find